

DBMS Assignment Answers

Q1A. Multiple Granularity Locking in DBMS

Multiple Granularity refers to the ability to lock data items at different levels of a hierarchy — from the entire database down to individual rows or fields — rather than being restricted to one fixed level.

The Granularity Hierarchy (Top to Bottom):

Database → Tables → Pages → Rows → Fields

This approach lets transactions choose the appropriate lock level: a transaction processing an entire table locks it at the table level, while one processing a few rows locks only those rows.

Intention Locks are used to signal intent at higher levels before locking at lower levels:

1. Intention Shared (IS): A transaction intends to place S-locks on some child nodes. Example: A transaction wants to read a few rows from a table — it places IS on the table, then S on specific rows.
2. Intention Exclusive (IX): A transaction intends to place X-locks on some child nodes. Example: A transaction wants to update certain rows — it places IX on the table, then X on specific rows.
3. Shared Intention Exclusive (SIX): The entire subtree is locked in S mode, but some children will also be locked in X mode. Example: A transaction reads the entire table but updates a few rows — it places SIX on the table.

Compatibility Matrix:

NLISIXSSIXXNL ✓✓✓✓✓✓ IS ✓✓✓✓✓ X IX ✓✓✓ XXXS ✓✓X XXSIX ✓✓XXXXX XXXXX

Rules:

Locking proceeds top-down (root to leaf).

Unlocking proceeds bottom-up (leaf to root).

Two-phase locking must be followed.

Example:

Transaction T1 wants to update Row 3 of Table A:

→ Place IX on Database → IX on Table A → X on Row 3

Transaction T2 wants to read all of Table B:

→ Place IS on Database → S on Table B

These two transactions can proceed concurrently without conflict since their lock paths don't collide.

Benefits: Flexibility and better concurrency — transactions don't over-lock.

Challenges: Added complexity in managing the lock hierarchy.

Q1B. Conflict Serializability and View Serializability

Conflict Serializability

Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a Write.

Conflicting pairs:

Read(A) by T1 and Write(A) by T2

Write(A) by T1 and Read(A) by T2

Write(A) by T1 and Write(A) by T2

A schedule is conflict serializable if it can be converted to a serial schedule by swapping only non-conflicting operations.

Testing using Precedence Graph:

Create a node for each transaction.

Draw an edge from $T_i \rightarrow T_j$ if an operation in T_i conflicts with and precedes an operation in T_j .

If the graph has no cycle, the schedule is conflict serializable.

Example:

T1T2Read(A)Read(A)Write(A)Read(B)Write(B)

Conflicts: W1(A) and R2(A) $\rightarrow$ edge $T_1 \rightarrow T_2$. No cycle. Conflict serializable (equivalent to $T_1 \rightarrow T_2$ serial order).

View Serializability

Two schedules S1 and S2 are view equivalent if:

Initial Read: If T_i reads the initial value of X in S1, it also does so in S2.

Updated Read: If T_i reads a value written by T_j in S1, the same holds in S2.

Final Write: If T_i writes the final value of X in S1, it does so in S2 too.

A schedule is view serializable if it is view equivalent to some serial schedule.

Example — Schedule S1:

T1T2Write(A)Read(A)Write(B)Read(B)Write(B)CommitCommit

If a serial schedule S2 satisfies all three view equivalence conditions with S1, then S1 is view serializable.

Key Difference: Every conflict serializable schedule is also view serializable, but not vice versa. View serializability is harder to test (NP-hard).

Q2. Cascading, Cascadeless, and Strict Schedules

1. Cascading Schedule (Cascading Rollback)

A cascading rollback occurs when one transaction's failure causes a chain of rollbacks in other transactions that read its uncommitted data.

Example:

T1T2Write(A)Read(A) ← dirty readWrite(B)AbortRollbackRollback ← cascades

T2 read uncommitted data from T1, so when T1 aborts, T2 must also be rolled back. This is wasteful and complex.

2. Cascadeless Schedule

A schedule is cascadeless if transactions only read committed values. A transaction can read a data item only after the transaction that last wrote it has committed.

Example:

T1T2Write(A)CommitRead(A) ← safeWrite(B)Commit

T2 reads A only after T1 commits, so no cascading rollback is possible.

3. Strict Schedule

A schedule is strict if no transaction reads or writes a data item written by an uncommitted transaction. This is stricter than cascadeless — it prevents both reads and writes on uncommitted data.

Example:

T1T2Write(A)CommitRead(A)Write(A)Commit

T2 neither reads nor writes A until T1 commits. Strict schedules simplify recovery significantly.

Relationship: Strict $\subset$ Cascadeless $\subset$ Recoverable

Q3A. Conflict Serializability vs. View Serializability

Feature	Conflict Serializability	View Serializability
Definition	Schedule convertible to serial by swapping non-conflicting ops	Schedule view equivalent to a serial schedule
Testing	Precedence graph (check for cycles)	NP-hard; no simple polynomial test
Strictness	Stricter	Subset
Generality	More general	Superset
Basis	Conflicting operations	Initial reads, updated reads, final writes
Relation	Every conflict serializable is view serializable	Not every view serializable is conflict serializable
Blind Writes	Not allowed	Allows blind writes (write without prior read)
Practicality	Used in practice	Mostly theoretical

Key insight: View serializability captures more valid schedules than conflict serializability, but is computationally infeasible to test in practice, so DBMSs use conflict serializability.

Q3B. Timestamp Ordering Protocol

In Timestamp-Based Protocols, each transaction T_i is assigned a unique timestamp $TS(T_i)$ when it begins (using system clock or a logical counter). Older transactions have smaller timestamps.

Each data item X maintains:

R-timestamp(X): Largest timestamp of any transaction that successfully read X .

W-timestamp(X): Largest timestamp of any transaction that successfully wrote X .

Read Rule: If T_i wants to Read(X):

If $TS(T_i) < W\text{-timestamp}(X) \rightarrow T_i$ is trying to read a value already overwritten by a newer transaction $\rightarrow$ Reject and rollback T_i .

Otherwise $\rightarrow$ Allow the read, update $R\text{-timestamp}(X) = \max(R\text{-timestamp}(X), TS(T_i))$.

Write Rule: If T_i wants to Write(X):

If $TS(T_i) < R\text{-timestamp}(X) \rightarrow$ A newer transaction already read X , so T_i 's write is too late $\rightarrow$ Reject and rollback T_i .

If $TS(T_i) < W\text{-timestamp}(X) \rightarrow$ A newer transaction already wrote $X \rightarrow$ Reject and rollback T_i .

Otherwise $\rightarrow$ Allow the write, update $W\text{-timestamp}(X) = TS(T_i)$.

Example:

Transactions T_1 ($TS=5$) and T_2 ($TS=10$):

T_2 writes $X \rightarrow W\text{-timestamp}(X) = 10$

T_1 tries to Read(X): $TS(T_1)=5 < W\text{-timestamp}(X)=10 \rightarrow T_1$'s read is rejected and T_1 is rolled back.

Another example:

T_1 reads $X \rightarrow R\text{-timestamp}(X) = 5$

T_2 writes $X \rightarrow W\text{-timestamp}(X) = 10$

T_1 tries to Write(X): $TS(T_1)=5 < W\text{-timestamp}(X)=10 \rightarrow T_1$'s write is rejected.

Advantages:

Deadlock-free (no locks involved).

Older transactions are given priority (fairness).

Better concurrency in many scenarios.

Disadvantages:

Starvation possible if a transaction is repeatedly rolled back.

Overhead of maintaining and comparing timestamps.

Cascading rollbacks can occur.

Q4A. Tree Indexes — Intuition and Advantages

Intuition behind Tree Indexes:

The core idea is to organize index entries in a hierarchical balanced tree structure (like B-Trees or B+ Trees). Each internal node contains key values that act as "signposts," guiding a search down toward the leaf nodes where the actual data or pointers to data reside.

Just like a binary search narrows down a search space by half each step, a tree index narrows it by a branching factor at each level — making retrieval logarithmic even for millions of records.

Structure of B+ Tree:

Internal nodes store keys and child pointers.

Leaf nodes store all key values and pointers to actual records.

Leaf nodes are linked in a chain → supports range queries efficiently.

Example B-Tree on Student IDs:

[1, 3]

/ \

[1] [3, 4]

/ \ / \

1 2 3 4

Navigating from root to leaf guides directly to the record.

Advantages of Tree-Based Indexing:

Sorted data: Keys are maintained in sorted order, making range queries (e.g., IDs between 100 and 200) very efficient.

Balanced: All root-to-leaf paths have equal length, so access time is consistently $O(\log n)$.

Multi-level: Minimizes disk I/Os by reducing search levels.

Dynamic: B+ Trees handle insertions and deletions without full reorganization — they split and merge nodes automatically.

Versatile: Efficient for both exact-match and range queries.

Disadvantages:

Slower than hash indexing for exact-match queries.

More complex to implement and maintain.

Q4B. Static Hashing — How It Works, Advantages, Disadvantages

How Static Hashing Works:

In static hashing, the data space is divided into a fixed number of buckets. A hash function $H(\text{key})$ maps a search key to a bucket number where the record is stored.

Steps:

A hash function takes the record's key as input.

It computes: $\text{bucket_number} = H(\text{key}) \bmod N$ (where N = number of buckets).

The record is stored in that bucket.

If a bucket is full, an overflow page is chained to it.

Example:

NameAgeSalaryAlice2850000Bob3560000Carol4070000

Hash Function: $H(x) = \text{ASCII value of first letter} \bmod 3$

Alice: $65 \bmod 3 = 2 \rightarrow \text{Bucket 2}$

Bob: $66 \bmod 3 = 0 \rightarrow \text{Bucket 0}$

Carol: $67 \bmod 3 = 1 \rightarrow \text{Bucket 1}$

Buckets:

Bucket 0 $\rightarrow$ Bob

Bucket 1 $\rightarrow$ Carol

Bucket 2 $\rightarrow$ Alice

To search for "Alice": compute $H(\text{"Alice"}) = 2$, go directly to Bucket 2 — very fast!

Advantages:

Extremely fast for exact-match (equality) queries — typically $O(1)$.

Simple to implement.

Well-suited for equality comparisons.

Disadvantages:

Not suitable for range queries (e.g., age between 20 and 30) — requires full scan.

If the number of records grows significantly, rehashing (expensive) is needed.

Poor hash functions lead to collisions and clustering, degrading performance.

Fixed number of buckets — cannot adapt dynamically (static limitation).

Q5. Primary and Secondary Indexing

Primary Index

A primary index is built on the primary key of an ordered data file. It is typically a sparse index — one entry per block (pointing to the first record of each block, called the anchor record).

Characteristics:

Based on the primary key (unique values).

Index entries are sorted in the same order as the data file.

One entry per disk block (sparse) or one per record (dense).

Example:

Students table:

RollNumber (PK)	Name	Age
1001	Madhu	20
1003	Mahi	22
1007	Ramu	21
1010	Durga	23

Assuming 2 records per block:

Block 1: RollNumbers 1001, 1003

Block 2: RollNumbers 1007, 1010

Sparse Primary Index:

Key (RollNumber)	Pointer
1001	Block 1
1007	Block 2

How it is used: To find RollNumber 1003, the DBMS looks at the index, finds that Block 1 covers 1001 onwards, fetches Block 1, and scans within it. Very fast retrieval.

Benefits: Fast key-based retrieval, enforces primary key uniqueness, efficient for range queries.

Secondary Index

A secondary index provides an alternative access path on non-primary-key attributes. It is always a dense index (entry for every record) and is non-clustered (physical order of data differs from index order).

Characteristics:

Built on non-primary-key attributes.

Can be non-unique (multiple records may share the same secondary key value).

Dense — one entry per record.

Points to the actual record location.

Example:

Using the same Students table, creating a secondary index on Age:

Dense Secondary Index on Age:

AgePointer to Record20Record 1 (RollNo 1001)21Record 3 (RollNo 1007)22Record 2 (RollNo 1003)23Record 4 (RollNo 1010)

How it is used: If a query asks "find all students aged 22", the DBMS looks up Age=22 in the secondary index, gets the pointer, and fetches the record directly — without scanning the whole table.

Benefits: Enables fast lookups on non-primary attributes, supports multiple access paths.

Drawback: Must be updated whenever data changes; consumes extra space.

Q6A. Closure Computation for $R(A, B, C, D, E, F, G)$

Given FDs:

$A \rightarrow BC$

$BC \rightarrow DE$

$D \rightarrow F$

$CF \rightarrow G$

Find the closure of A, i.e., A^+

Step 1: Start with $A^+ = \{A\}$

Step 2: Apply $A \rightarrow BC$

$\rightarrow A^+ = \{A, B, C\}$

Step 3: Apply $BC \rightarrow DE$ ($BC \subseteq A^+$)

$\rightarrow A^+ = \{A, B, C, D, E\}$

Step 4: Apply $D \rightarrow F$ ($D \subseteq A^+$)

$\rightarrow A^+ = \{A, B, C, D, E, F\}$

Step 5: Apply $CF \rightarrow G$ ($C, F \subseteq A^+$)

$\rightarrow A^+ = \{A, B, C, D, E, F, G\}$

No more FDs applicable.

Result: $A^+ = \{A, B, C, D, E, F, G\}$

Since A^+ contains all attributes of R , A is a candidate key for the relation $R(A, B, C, D, E, F, G)$.

Q6B. Differences Between BCNF and 3NF

Feature	3NF (Third Normal Form)	BCNF (Boyce-Codd Normal Form)
Definition	No transitive dependencies; non-key attributes depend only on the primary key	For every non-trivial FD $X \rightarrow Y$, X must be a superkey
Strictness	Less strict	More strict ($3NF \subset BCNF$)
Dependency Preservation	Always preserves all functional dependencies	May not always preserve all functional dependencies
Lossless Decomposition	Always possible	Always possible
Anomalies	May still have some redundancy	Eliminates more redundancy than 3NF
Transitive Dependencies	Not allowed	Not allowed
Partial Dependencies	Not allowed	Not allowed
Exception	Allows FD $X \rightarrow Y$ where Y is a prime attribute (part of candidate key)	No such exception; X must always be a superkey
Practical Use	Preferred when dependency preservation is critical	Preferred when redundancy elimination is the priority

Key Insight:

In 3NF, there is one exception: a non-superkey X can still determine Y if Y is a prime attribute (part of some candidate key).

BCNF removes this exception entirely — every determinant must be a superkey.

As a result, every BCNF schema is in 3NF, but not every 3NF schema is in BCNF.

Example of a schema in 3NF but NOT in BCNF:

Relation: $R(A, B, C)$ with FDs: $A \rightarrow B$, $BC \rightarrow A$

Candidate keys: $\{A, C\}$ and $\{BC\}$. The FD $BC \rightarrow A$ has BC as a superkey, but $A \rightarrow B$ does not have A as a superkey (since A alone is not a superkey... wait — A is a candidate key here, so this particular example is in BCNF too). A classic case would be where a non-superkey non-prime attribute dependency exists — BCNF would require decomposition while 3NF would still accept it.